Perfect Uttam 🤲 — you're thinking absolutely in the right *real-world* direction.
This part (order workflow between **User → Vendor → Admin**) is the *core* of every e-commerce platform — so let's design this **practically and logically** so that your backend, database, and APIs work smoothly together.

🧩 Step-by-Step Order Flow (User → Vendor → Admin)

🛍️ 1️⃣ User places an order

Tables involved:

- users
- products
- orders
- order_details

Flow:

1. User selects one or more products and clicks **"Place Order."**
2. A new record is created in the **orders** table:
3. orders (
4. order_id INT PK,
5. user_id FK,
6. vendor_id FK,
7. order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
8. total_amount DECIMAL(10,2),
9. delivery_status ENUM('Pending','Processing','Shipped','Delivered','Cancelled') DEFAULT 'Pending',
10. payment_status ENUM('Pending','Paid','Failed') DEFAULT 'Pending'
11.)
12. Each item in the user's cart is stored in the **order_details** table:

13. order_details (
14. order_detail_id INT PK,
15. order_id FK,
16. product_id FK,
17. quantity INT,
18. price DECIMAL(10,2),
19. subtotal DECIMAL(10,2)
20.)
21. When the order is created, the system automatically links:
    - user_id → from the logged-in user
    - vendor_id → from each product (since each product belongs to one vendor)

💳 2️⃣ Payment and Commission Split (API logic)

When payment is successful:

1. The **payment API** receives the total order amount.
   Example: ₹10,000
2. The system checks the vendor's commission type:
    - If **commission-based**, calculate:
    - adminCommission = totalAmount * 0.05; // 5%
    - vendorAmount = totalAmount - adminCommission;
3. Update **payment_status = 'Paid'** in orders.
4. Insert records into a new table:
5. transactions (
6. transaction_id INT PK,
7. order_id FK,
8. vendor_id FK,

9. admin_id FK,
10. total_amount DECIMAL(10,2),
11. vendor_share DECIMAL(10,2),
12. admin_commission DECIMAL(10,2),
13. payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14.)

💡 **At this point:**

- Admin automatically gets 5% commission logged.
- Vendor balance (virtual or wallet) is updated with 95%.

🚚 3️⃣ **Vendor gets notified of a new order**

There are two main ways to notify vendors:

1. **WebSocket or Event-driven notification:**
    - When an order is created, trigger a WebSocket event to the vendor dashboard (e.g., NEW_ORDER_RECEIVED).
2. **Backend query filtering:**
    - Vendor dashboard fetches:
    - SELECT * FROM orders WHERE vendor_id = ? AND delivery_status != 'Delivered';

Vendor can see:

- Order details
- Buyer info (limited: name, address, contact)
- Payment status
- Delivery status

📦 4️⃣ **Vendor updates the delivery status**

Vendor changes the order's delivery_status:

- Pending → Processing → Shipped → Delivered

This is updated via API:

PUT /api/vendor/orders/:orderId/status

{

"status": "Delivered"

}

→ updates orders.delivery_status
→ triggers WebSocket event to the **user** to show *"Your product has been delivered."*

📜 5️⃣ Order history for user dashboard

User dashboard fetches:

SELECT o.order_id, o.order_date, o.delivery_status, od.product_id, od.quantity, od.price

FROM orders o

JOIN order_details od ON o.order_id = od.order_id

WHERE o.user_id = ?;

Shows:

- Product list
- Status
- Payment info
- Delivery updates

This becomes their **order history page**.

📊 6️⃣ Vendor's sales report

Vendor dashboard queries:

SELECT

SUM(od.subtotal) AS total_sales,

```
    COUNT(o.order_id) AS total_orders,
    SUM(t.admin_commission) AS total_commission_paid
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN transactions t ON o.order_id = t.order_id
WHERE o.vendor_id = ? AND o.delivery_status = 'Delivered';
```

This gives:

- Total revenue
- Commission paid to admin
- Net earnings

👨‍💼 7️⃣ Admin monitoring

Admin dashboard queries:

```
SELECT
    v.vendor_id,
    v.vendor_name,
    SUM(t.admin_commission) AS total_commission,
    SUM(t.vendor_share) AS total_vendor_payout
FROM transactions t
JOIN vendors v ON v.vendor_id = t.vendor_id
GROUP BY v.vendor_id;
```

Admin sees:

- Each vendor’s commission total
- Monthly payouts
- Overall revenue growth

✅ **Summary of Flow**

Step	Actor	Action	Table(s) Involved
1	User	Place order	orders, order_details
2	Payment API	Split amount (95% vendor / 5% admin)	transactions
3	Vendor	Get notified of new order	orders
4	Vendor	Update delivery status	orders
5	User	See order history	orders, order_details
6	Vendor	View sales report	transactions
7	Admin	Monitor vendors & commissions	transactions, vendors